TALLER DE PROGRAMACIÓN ORIENTADA A OBJETOS

Relaciones entre clases en Java

Estudiante: [Escribe tu nombre completo]

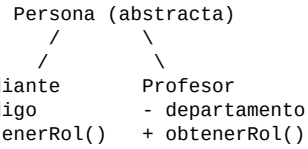
Asignatura: Programación Orientada a Objetos

Universidad: [Escribe el nombre de tu universidad]

Repositorio de GitHub: [Pega aquí la URL después de subir el proyecto]

1. Herencia y diagrama

La herencia permite crear una clase a partir de otra. La clase hija reutiliza atributos y métodos de la clase padre y puede agregar características propias o redefinir métodos. En este ejemplo, Estudiante y Profesor heredan de Persona.



Persona concentra los datos comunes: nombre y documento. Estudiante añade un código y Profesor añade un departamento. Ambos implementan obtenerRol() de manera diferente.

Código representativo — Persona.java

```
public abstract class Persona {
    protected String nombre;
    protected String documento;

    public Persona(String nombre, String documento) {
        this.nombre = nombre;
        this.documento = documento;
    }

    public String getNombre() {
        return nombre;
    }

    public String getDocumento() {
        return documento;
    }

    // Metodo abstracto: cada tipo de persona define su propio rol.
    public abstract String obtenerRol();
}
```

2. Diferencia entre clase abstracta e interfaz

Clase abstracta	Interfaz
Puede tener atributos de instancia, constructores y métodos implementados además de abstractos.	Define un contrato de métodos que las clases implementadoras deben cumplir; tamb
Una clase puede extender solamente una clase.	Una clase puede implementar varias interfaces.
Se usa para compartir estado y comportamiento común entre clases relacionadas.	Se usa para establecer capacidades o comportamientos que distintas clases pueden

En el proyecto, Persona es una clase abstracta porque representa una idea general y declara obtenerRol(). Calificable es una interfaz porque establece que una clase debe ofrecer el método calcularNotaFinal().

3. Ejemplo de dependencia

Existe dependencia cuando una clase utiliza otra para realizar una operación, por ejemplo, cuando recibe un objeto como parámetro de un método. ReporteAcademico depende de Estudiante porque usa sus datos para imprimir el reporte. El vínculo se utiliza dentro de la operación y no representa por sí mismo una relación permanente entre ambos objetos.

```
public void generarReporte(Estudiante estudiante) {
    System.out.println(estudiante.getNombre());
    System.out.println(estudiante.getCodigo());
}
```

4. Asociación muchos a muchos

Un estudiante puede matricularse en varios cursos y cada curso puede tener varios estudiantes. Por eso la relación entre Estudiante y Curso es muchos a muchos. El proyecto representa cada inscripción mediante la clase asociativa Matricula, que guarda una referencia al estudiante y otra al curso.

```
Estudiante  1 ---- 0..* Matricula 0..* ---- 1 Curso

Ejemplo:
Laura -> Programacion en Java
Laura -> Matematicas
Andres -> Programacion en Java
```

La clase Matricula hace explícita cada inscripción y permite representar la relación entre ambos lados.

5. Ejemplo de interfaz

Calificable es una interfaz con el método calcularNotaFinal(). La clase EstudianteCalificable la implementa y calcula una nota ponderada usando tres notas: 35 %, 35 % y 30 %. Así, la clase cumple el contrato definido por la interfaz.

```
public interface Calificable {
    // Cualquier clase que implemente esta interfaz debe definir este metodo.
    double calcularNotaFinal();
}

public class EstudianteCalificable implements Calificable {
    private String nombre;
    private double nota1;
    private double nota2;
    private double nota3;

    public EstudianteCalificable(String nombre, double nota1, double nota2, double nota3) {
        this.nombre = nombre;
        this.nota1 = nota1;
        this.nota2 = nota2;
        this.nota3 = nota3;
    }

    public String getNombre() {
        return nombre;
    }

    @Override
    public double calcularNotaFinal() {
        // Ejemplo de ponderacion: 35 %, 35 % y 30 %.
        return nota1 * 0.35 + nota2 * 0.35 + nota3 * 0.30;
    }
}
```

6. Cómo ejecutar el proyecto

1. Crear un proyecto Java en NetBeans llamado TallerPOORelaciones. 2. Copiar los archivos de la carpeta src al paquete principal. 3. Si NetBeans usa una declaración package, ponerla al inicio de todos los archivos. 4. Ejecutar Main.java y verificar que se muestren los ejemplos de herencia, interfaz, dependencia y matrículas.

7. Repositorio

Enlace de GitHub: [Pega aquí la URL real del repositorio después de publicarlo]. No entregues el documento con este marcador sin reemplazarlo por el enlace real.